

Initial Setup On VMware Ubuntu

- **Update System**

sudo apt update

Install Required Tools

- **Install Git**

sudo apt install git -y

- **Install Docker**

sudo apt install docker.io -y

- **Start Docker**

sudo systemctl start docker

sudo systemctl enable docker

- **Check Docker**

docker --version

- **Install Ansible**

sudo apt install ansible -y

- **Check Ansible**

ansible --version

Q1. Write a playbook (Directory-playbook.yml) to create a directory `/tmp/testdir` on all managed nodes. Delete directory after verifying its creation.

Step 1: Create Inventory File

```
mkdir devops
```

```
cd devops
```

```
nano inventory
```

Add:

```
[servers]
```

```
localhost ansible_connection=local
```

Save:

```
CTRL + O
```

```
ENTER
```

```
CTRL + X
```

Step 2: Create Playbook

```
nano Directory-playbook.yml
```

Add:

```
---
```

```
- hosts: servers
```

```
  become: yes
```

```
tasks:
```

```
- name: Create directory
```

```
  file:
```

```
    path: /tmp/testdir
```

```
    state: directory
```

```
- name: Verify directory
```

```
  shell: ls /tmp
```

register: output

- debug:

var: output.stdout_lines

- name: Delete directory

file:

path: /tmp/testdir

state: absent

Step 3: Run Playbook

ansible-playbook -i inventory Directory-playbook.yml

Step 4: Validate

ls/temp

Q2. Write a simple application. Build & share the image to Docker Hub.

Step 1: Create Project

```
mkdir dockerapp  
cd dockerapp
```

Step 2: Create Python File

```
nano app.py
```

Add:

```
print("Hello Docker")
```

Step 3: Create DockerFile

```
nano Dockerfile
```

Add:

```
FROM python:3  
COPY app.py /app.py  
CMD ["python","/app.py"]
```

Step 4: Build Image

```
sudo docker build -t yourdockerhubusername/myapp:v1 .
```

Step 5: Run Image

```
sudo docker run yourdockerhubusername/myapp:v1
```

Step 6: Login Docker Hub

```
sudo docker login
```

Enter:

DockerHub Username

DockerHub Password

Step 7: Push Image

```
sudo docker push yourdockerhubusername/myapp:v1
```

Q3. Write GIT commands to:**A. Clone a remote repository**

git clone <https://github.com/user/repository.git>

B. Create new branch DEV

git checkout -b DEV

C. Modify file & commit changes

nano file.txt

Then:

git add file.txt

git commit -m "Modified file"

D. Merge into main

git checkout main

git merge DEV

E. Push main branch to remote repository

git push origin main

Q4. Describe the steps to resolve the merge conflict when two merge. Include GIT Commands to:

A. Identify conflicting file

Step 1: Check Conflict

git status

Step 2: Open File

nano filename

Remove:

<<<<<<< HEAD

=====

>>>>>>> DEV

Step 3: Add File

git add filename

Step 4: Commit

git commit -m "Resolved merge conflict"

Q5. Write GIT Commands to:**A. Create a local GIT repository named as Project Alpha****Step 1: Create Folder**

```
mkdir ProjectAlpha  
cd ProjectAlpha
```

Step 2: Initialize GIT

```
git init
```

B. Add 2 files notes.txt(with content “Meeting Agenda”) and data.csv(Empty)**Create Files:**

```
echo "Meeting Agenda" > notes.txt  
touch data.csv
```

Check:

```
ls
```

C. Commit these files with initial

```
git add .  
git commit -m "Initial commit"
```

D. Create a branch experiment & add a file [test.py](#) with print hello. Commit with the message “Experimental feature”**Step 1: Create Branch**

```
git checkout -b experiment
```

Step 2: Create File

```
nano test.py
```

Add:

```
print("hello")
```


Step 3: Commit

```
git add test.py
```

```
git commit -m "Experimental feature"
```

Q6. Configure a CI/CD pipeline file to include a schedule job that runs every minute using GIT lab scheduling feature. Verify the pipeline execution by checking

Step 1: Create .gitlab-ci.yml

```
nano .gitlab-ci.yml
```

Add:

```
stages:
```

```
- test
```

```
job1:
```

```
  stage: test
```

```
  script:
```

```
    - echo "Pipeline Running"
```

Step 2: Push Code

```
git add .
```

```
git commit -m "Added pipeline"
```

```
git push origin main
```

Step 3: Schedule Pipeline

Open Browser inside VMWare

Go to:

GitLab → Your Project → Build → Pipeline Schedules

Set:

```
* * * * *
```

Q7. Use docker to modify an existing image, commit and save it as a new one**Step 1: Pull Ubuntu Image**

```
sudo docker pull ubuntu
```

Step 2: Run Container

```
sudo docker run -it ubuntu bash
```

Step 3: Modify Container**Inside container:**

```
apt update  
touch sample.txt
```

Exit:

```
exit
```

Step 4: Check Container ID

```
sudo docker ps -a
```

Step 5: Commit Container

```
sudo docker commit <container_id> myubuntu:v1
```

Step 6: Verify

```
sudo docker images
```

Q9. Perform following ansible actions:**A. Create & Delete directory manage node**

- Create directory

```
ansible localhost -m file -a "path=/tmp/demo state=directory" -c local
```

- Delete directory

```
ansible localhost -m file -a "path=/tmp/demo state=absent" -c local
```

B. Copy files to remote host using adhoc commands**Create file:**

```
echo "Hello" > test.txt
```

Copy:

```
ansible localhost -m copy -a "src=test.txt dest=/tmp/test.txt" -c local
```

Q10. Set up new project in GIT lab. Add CI/CD file to define a basic pipeline with stages like build and wind. Push changes to repository and verify that the pipeline executes successfully

Step 1: Create GITLab repository

Open Browser:

GitLab → New Project

Step 2: Create .gitlab-ci.yml

nano .gitlab-ci.yml

Add:

```
stages:
- build

build_job:
  stage: build
  script:
  - echo "Build Successful"
```

Step 3: Push Code

```
git add .
git commit -m "Added CI/CD"
git push origin main
```

Step 4: Verify Pipeline

Go to:

GitLab → CI/CD → Pipelines

Green tick means successful

Q11. Deploy an HTML page using GIT Lab CI/CD**Step 1: Create HTML File**

```
nano index.html
```

Add:

```
<html>
<head>
<title>My Page</title>
</head>

<body>
<h1>Hello DevOps</h1>
</body>
</html>
```

Step 2: Create .gitlab-ci.yml

```
nano .gitlab-ci.yml
```

Add:

```
pages:
script:
- mkdir public
- cp index.html public/
artifacts:
paths:
- public
```

Step 3: Push Files

```
git add .
git commit -m "Deploy HTML page"
git push origin main
```

Step 4: View Website**Go To:**

GitLab → Deploy → Pages

Q12. Write a playbook (Ping_playbook.yml) that uses the ping module to test connective for all nodes in your inventory file. Run the playbook and confirm that all nodes respond successfully.

Step 1: Create Playbook

```
nano Ping-playbook.yml
```

Add:

```
---
```

```
- hosts: localhost
```

```
tasks:
```

```
- name: Ping test
```

```
ping:
```

Step 2: Run Playbook

```
ansible-playbook Ping-playbook.yml
```

Q13. Write a simple application to create an image container the image, push it to docker hub and then remove the images using docker.

Step 1: Create App

```
mkdir app1  
cd app1
```

Step 2: Create HTML File

```
nano index.html
```

Add:

```
<h1>Hello Docker</h1>
```

Step 3: Create Dockerfile

```
nano Dockerfile
```

Add:

```
FROM nginx  
COPY index.html /usr/share/nginx/html/index.html
```

Step 4: Build Image

```
sudo docker build -t yourdockerhubusername/webapp:v1 .
```

Step 5: Run Container

```
sudo docker run -d -p 8080:80 yourdockerhubusername/webapp:v1
```

Step 6: Push to Docker Hub

```
sudo docker login  
sudo docker push yourdockerhubusername/webapp:v1
```

Step 7: Remove Image

```
sudo docker rmi yourdockerhubusername/webapp:v1
```


Q14. Execute any 5 docker commands and explain their outputs.

1. Check Docker Version

`docker --version`

Output:

Shows installed Docker version.

2. Pull Image

`docker pull nginx`

Output:

Downloads nginx image from Docker Hub.

3. View Images

`docker images`

Output:

Displays downloaded Docker images.

4. Run Container

`docker run -d nginx`

Output:

Starts nginx container in background.

5. List Running Containers

`docker ps`

Output:

Shows running containers.\

Q15. Deploy a HTML webpage using a docker file and share it an docker hub.

Step 1: Create Folder

```
mkdir webpage  
cd webpage
```

Step 2: Create HTML File

```
nano index.html  
Add:  
    <h1>My Docker Website</h1>
```

Step 3: Create Dockerfile

```
nano Dockerfile  
  
Add:  
    FROM nginx  
    COPY index.html /usr/share/nginx/html/index.html
```

Step 4: Build Image

```
sudo docker build -t yourdockerhubusername/htmlpage:v1 .
```

Step 5: Run Container

```
sudo docker build -t yourdockerhubusername/htmlpage:v1 .
```

Step 6: Push to Docker Hub

```
sudo docker login  
sudo docker push yourdockerhubusername/htmlpage:v1
```

Q16. Write GIT commands to clone remote repository make changes, merge and commit to a new branch on remote repository.

Step 1: Clone Repository

```
git clone https://github.com/user/repository.git
```

Step 2: Enter Repository

```
cd repository
```

Step 3: Create NewBranch

```
git checkout -b newbranch
```

Step 4: Make Changes

```
nano file.txt
```

Step 5: Add & Commit

```
git add .  
git commit -m "Updated file"
```

Step 6: Push Branch

```
git push origin newbranch
```